

In-Class Assignment 4: LiDAR Spoofing Attacks and Defense

Guest Lecture by Charles M. Raines

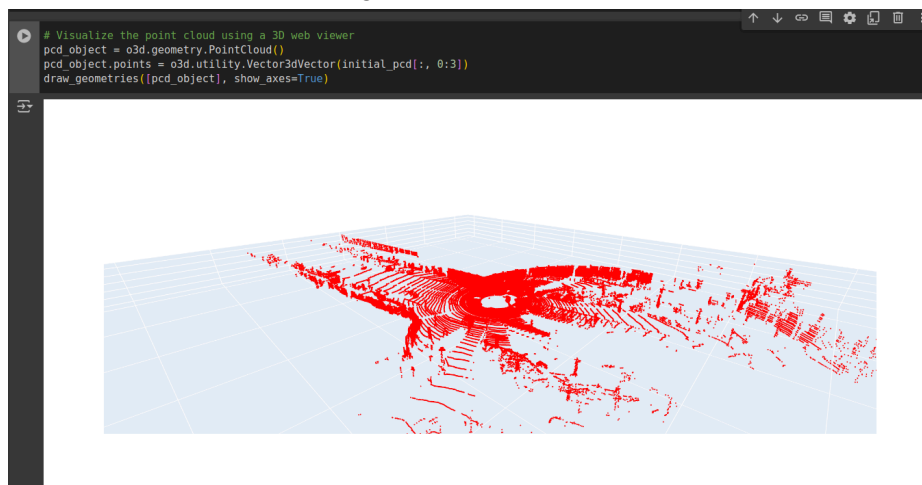
This notebook is an in-class assignment on the topic of LiDAR attacks and defense in robotic systems. We will go over and implement two different types of LiDAR attacks, Injection and Laser Spoofing, and discuss a trivial defense strategy: density filtering.

0. Imports and Loading Google Drive

Before completing the assignment, we must first import the proper libraries and connect our google drive. First execute the first block to install the Open3D Library. Next, you need to import the contents of the InClass4.zip folder on canvas into your google colab session. To do this, upload the folder contents to your google drive. Next, change the path in the last two lines of code in block two to match the path of the contents in your google drive. For example, if your contents are in a folder named "in_class_4", the path would most likely be `"/content/drive/MyDrive/in_class_4/"`. Finally, run code block three to import the required libraries.

1. Point Cloud Data Loading and Visualization

Using the `np.loadtxt()` function, load the `cloud.txt` file from your google drive directory. Append `.astype(np.float32)` to the end of the code statement to ensure all values are loaded in as a 32 bit floating point value. Run the block of code. Run the next block of code to visualize the pointcloud data and confirm we have properly loaded the data in. If done correctly, the output should look like the following:



2. Point Cloud Filtering

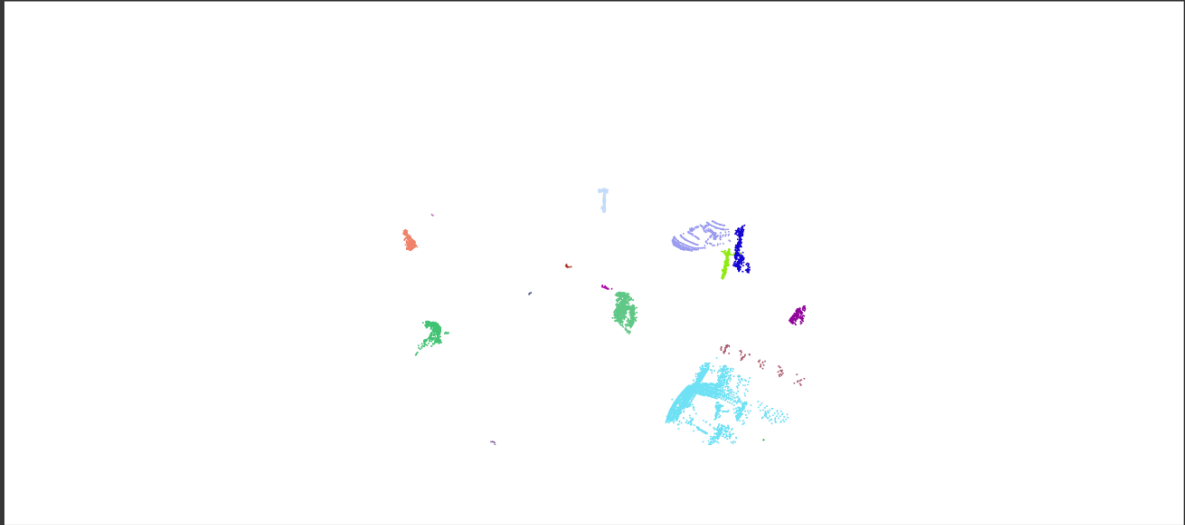
Simply run all of the blocks of code in this section to re-create the LiDAR processing you did in Lab 2 of this course. This will help us simplify our work in subsequent sections. When finished, the output should look identical to the output at the end of Lab 2:

```
[ ] # Perform Clustering on Point Cloud
```

```
cluster_labels = euclidean_clustering(pcd_filtered_2)

print('Found %d clusters from %d points'%(cluster_labels.max(), len(pcd_filtered_2)))
pcd_object.points = o3d.utility.Vector3dVector(pcd_filtered_2)
color_sample_state = np.random.RandomState(0)
obj_color = color_sample_state.random((np.max(cluster_labels)+1,3))
pcd_object.colors = o3d.utility.Vector3dVector(obj_color[cluster_labels])
draw_geometries([pcd_object])
```

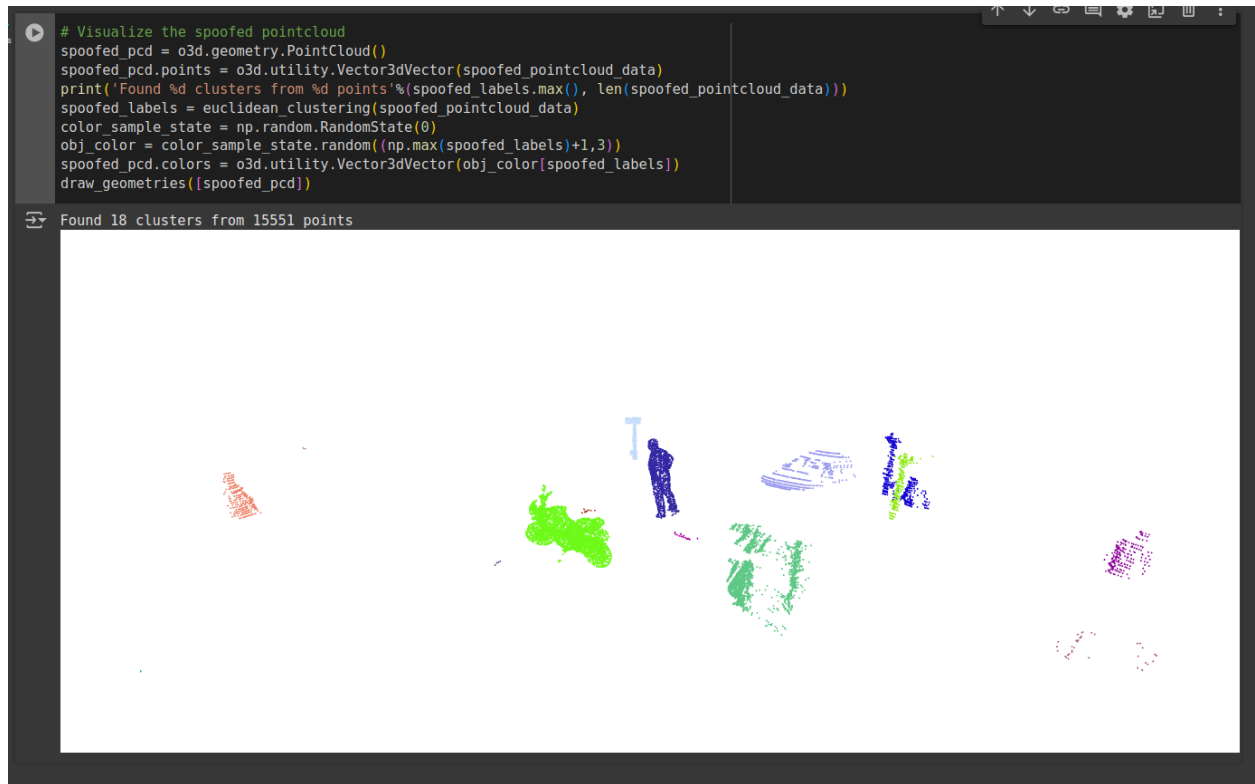
```
Found 16 clusters from 9049 points
```



3. PointCloud Data Injection

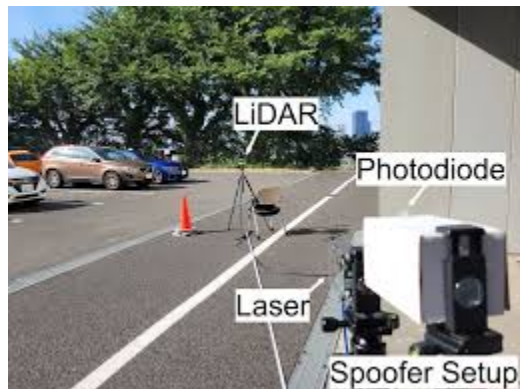
For this implementation section, we will be injecting false LiDAR data into the pointcloud we currently have open. Each step is listed in a green comment. Follow each step in green in order to complete the section.

(HINT: For the steps that utilize the `np.vstack()` function, keep in mind this function expects a tuple as the argument.) The final output should look like this:

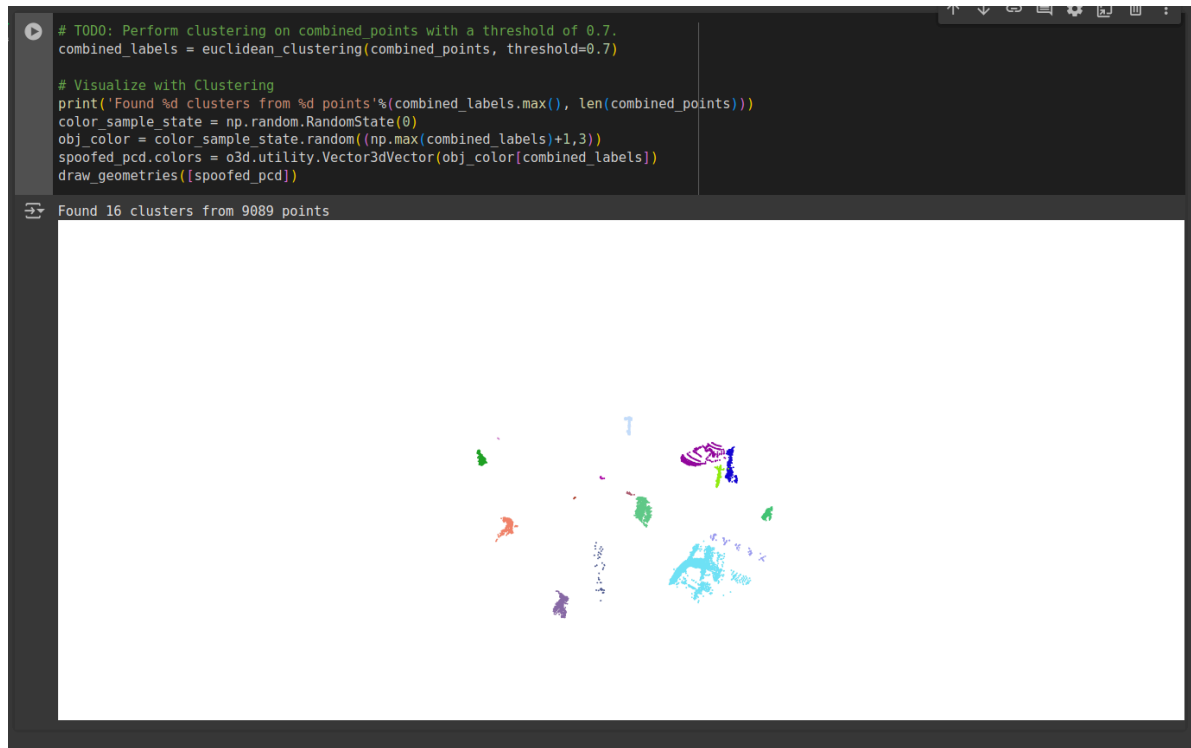


4. Simulating a Laser Spoofer

For this section, we will be simulating the effects of a laser spoofer on point cloud data.



Above is an image of a laser spoofer. This device detects the pulses of the LiDAR and, in carefully timed intervals, emits its own lasers to trick the LiDAR, resulting in spoofed points. This section of code will simulate those effects. Follow the green comments in the code blocks to implement the `generate_spoofed_laser_points()` function and successfully simulate a laser spoofer. The final output should look like this:

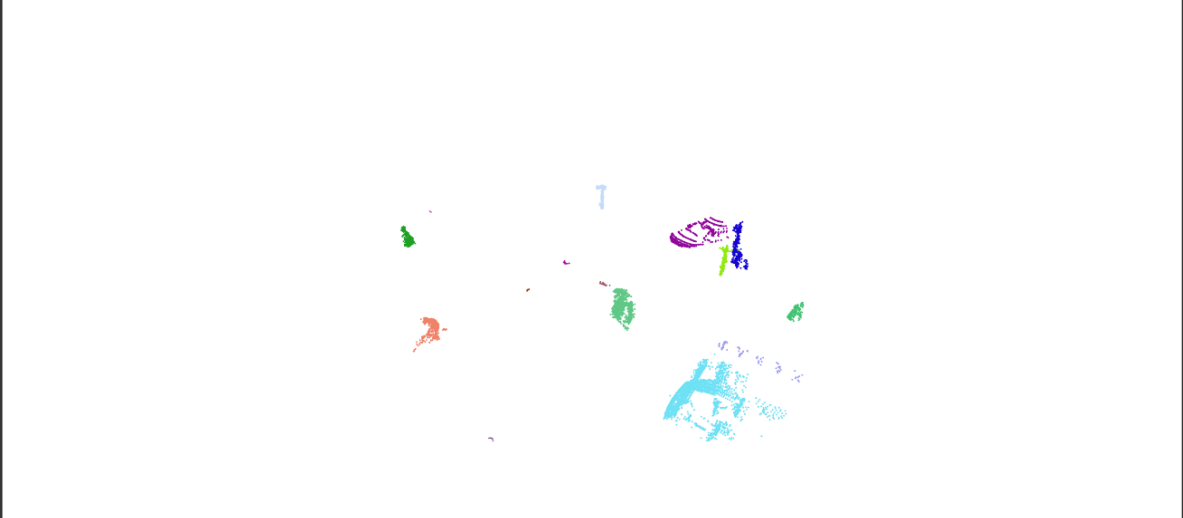


5. Defense Against Laser Spoofer

A trivial defense against a spoofing attack would be to identify clusters that are unique in density compared to the other clusters and filter those clusters out of the pointcloud. For this section, follow the green comments to implement a filtering function, `filter_spoofed_points()`, and successfully defend against a LiDAR spoofing attack. The final result should show the absence of spoofed points in the point cloud:

```
[ ] # Visualize the filtered pointcloud data with no spoofed points
print('Found %d clusters from %d points'%(filtered_labels.max(), len(filtered_cloud)))
filtered_pcd = o3d.geometry.PointCloud()
filtered_pcd.points = o3d.utility.Vector3dVector(filtered_cloud)
color_sample_state = np.random.RandomState(0)
obj_color = color_sample_state.random((np.max(filtered_labels)+1,3))
filtered_pcd.colors = o3d.utility.Vector3dVector(obj_color[filtered_labels])
draw_geometries([filtered_pcd])
```

Found 15 clusters from 9049 points



Deliverables

Submit the completed google colab notebook on canvas as an attachment.